

Complex

复数

如果您已经学习过复数相关知识，请跳过本页面。

学习复数知识需要一部分向量基础，如果并未学习过向量知识请移步 [向量页面](#)。

复数

引入

▼ 注

下面的引入方法来自人教版高中数学 A 版必修二。

从方程的角度看，负实数能不能开平方，就是方程 $x^2 = -1$ 有没有解，进而可以归结为方程 $x^2 + 1 = 0$ 有没有解。

回顾已有的数集扩充过程，可以看到，每次扩充都与实际需求密切相关。例如，为了解决正方形对角线的度量，以及这样的方程在有理数集中无解的问题，人们把有理数集扩充到了实数集。数集扩充后，在实数集中规定的加法运算、乘法运算，与原来在有理数集中规定的加法运算、乘法运算协调一致，并且加法和乘法都满足交换律和结合律，乘法对加法满足分配律。

依照这种思想，为了解决 $x^2 + 1 = 0$ 这样的方程在实数系中无解的问题，我们设想引入一个新数 i ，使得 i 是方程 $x^2 + 1 = 0$ 的解，即使得 $i^2 = -1$ 。

思考：把新引进的数 i 添加到实数集中，我们希望数 i 和实数之间仍然能像实数那样进行加法和乘法运算，并希望加法和乘法都满足交换律、结合律，以及乘法对加法满足分配律。那么，实数系经过扩充后，得到的新数系由哪些数组成呢？

依照以上设想，把实数 a 与 i 相乘，结果记作 ai ；把实数 a 与 i 相加，结果记作 $a + i$ 。注意到所有实数以及 ai 都可以写成 $a + bi$ 的形式，从而这些数都在扩充后的新数集中。

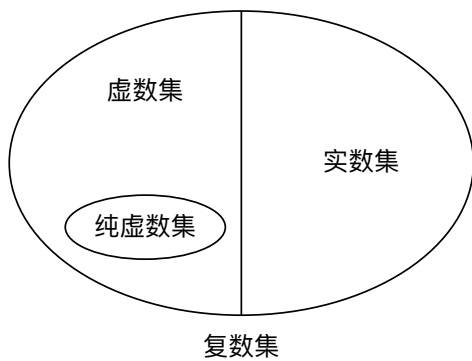
定义

我们定义形如 $a + bi$ ，其中 $a, b \in \mathbb{R}$ 的数叫做 **复数**，其中 i 被称为 **虚数单位**，全体复数的集合叫做 **复数集**，记作 $\mathbb{C}$ 。

复数通常用 z 表示，即 $z = a + bi$ 。这种形式被称为 **复数的代数形式**。其中 a 称为复数 z 的 **实部**，记作 $\operatorname{Re} z$ ， b 称为复数 z 的 **虚部**，记作 $\operatorname{Im} z$ 。如无特殊说明，都有 $i^2 = -1$ 。

对于一个复数 $z = a + bi$ ，当且仅当 $b = 0$ 时，它是实数，当 $b \neq 0$ 时，它是虚数，当 $a = 0$ 且 $b \neq 0$ 时，它是纯虚数。

纯虚数，虚数，实数，复数的关系如下图所示。



性质与运算

几何意义

我们知道了 这样类似的形式为数被称为复数，并且给出了定义和分类，我们还可以挖掘一下更深层的性质。

我们把所有实数都放在了数轴上，并且发现数轴上的点与实数一一对应。我们考虑对复数也这样处理。

首先我们定义 **复数相等**：两个复数 是相等的，当且仅当 且 。

这么定义是十分自然的，在此不做过多解释。

也就是说，我们可以用唯一的有序实数对 表示一个复数 。这样，联想到平面直角坐标系，我们可以发现 **复数集与平面直角坐标系中的点集一一对应**。好了，我们找到了复数的一种几何意义。

那么这个平面直角坐标系就不再一般，因为平面直角坐标系中的点具有了特殊意义——表示一个复数，所以我们把这样的平面直角坐标系称为 **复平面**， 轴称为 **实轴**， 轴称为 **虚轴**。我们进一步地说：**复数集与复平面内所有的点所构成的集合是一一对应的**。

我们考虑到学过的平面向量的知识，发现向量的坐标表示也是一个有序实数对，显然，复数 对应复平面内的点，那么它还对应平面向量，于是我们又找到了复数的另一种几何意义：**复数集与复平面内的向量所构成的集合是一一对应的（实数 与零向量对应）**。

于是，我们由向量的知识迁移到复数上来，定义 **复数的模** 就是复数所对应的向量的模。复数 的模 。

于是为了方便，我们常把复数 称为点 或向量，并规定相等的向量表示同一个复数。

并且由向量的知识我们发现，虚数不可以比较大小（但是实数是可以的）。

加法与减法

对复数，定义加法规则如下：

很明显，两个复数的和仍为复数。

考虑到向量的加法运算，我们发现复数的加法运算符合向量的加法运算法则，这同样证明了复数的几何意义的正确性。

同样可以验证，复数的加法满足 **交换律** 和 **结合律**。即：

减法作为加法的逆运算，我们可以通过加法法则与复数相等的定义来推导出减法法则：

这同样符合向量的减法运算。

乘法、除法与共轭

对复数 z ，定义乘法规则如下：

可以看出，两个复数相乘类似于两个多项式相乘，只需要把 i^2 换成 -1 ，并将实部与虚部分别合并即可。

复数的乘法与向量的向量积形式类似。

易得复数乘法满足 **交换律**，**结合律** 和 **对加法的分配律**，即：

-
-
-

由于满足运算律，我们可以发现实数域中的 **乘法公式在复数域中同样适用**。

除法运算是乘法运算的逆运算，我们可以推导一下：

由于向量没有除法，这里不讨论与向量的关系。

为了分母实数化，我们乘了一个 $\bar{z}$ ，这个式子很有意义。

对复数 z ，称为 $\bar{z}$ 的 **共轭复数**，通常记为 $\bar{z}$ 。我们可以发现，若两个复数互为共轭复数，那么它们 **关于实轴对称**。

对复数 z ，复数共轭有如下性质

-
-
- $\overline{z_1 + z_2} = \bar{z}_1 + \bar{z}_2$
-
-
-

辐角和辐角主值

如果设定实数单位 1 作为水平正方向，虚数单位 i 作为竖直正方向，得到的就是直角坐标视角下的复平面。

表示复数 z 的位置，也可以借助于极坐标 确定。前文已经提到了 $|z|$ 为复数 z 的模。

从实轴正向到 **非零** 复数 z 对应向量的夹角 θ 满足关系：

称为复数 z 的 **辐角**，记为：

任一个 **非零** 复数 z 有无穷多个辐角，故 事实上是一个集合。借助开头小写的 θ 表示 **其中一个特定值**，满足条件：

称为 **辐角主值** 或 **主辐角**。辐角就是辐角主值基础上加若干整数个（可以为零或负整数），即 $\theta + 2k\pi$ 。

需要注意的是两个辐角主值相加后不一定还是辐角主值，而两个辐角相加一定还是合法的辐角。

称模小于 1 的复数，在复平面上构成的图形为 **单位圆**。称模等于 1 的复数为 **单位复数**，全体单位复数在复平面上构成的

图形为 **单位圆周**。在不引起混淆的情况下，有时单位圆周也简称单位圆。

在极坐标的视角下，复数的乘除法变得很简单。复数乘法，模相乘，辐角相加。复数除法，模相除，辐角相减。

欧拉公式

▼ 欧拉公式 (Euler's formula) ¹

对任意实数，有

在补充 [复指数函数与复三角函数](#) 的定义后，该公式可推广至全体复数。

指数函数与三角函数

对于复数，函数 满足。由此给出 **复指数函数** 的定义：

复指数函数在实数集上与实指数函数的定义完全一致。在复平面上拥有性质：

- 模恒正：。
- 辐角主值：。
- 加法定理：。
- 周期性：是以 为基本周期的周期函数。如果一个函数 的周期是某一周期的整倍数，称该周期为 **基本周期**。

复三角函数（也简称 **三角函数**）的定义如下：

若取，则由 [欧拉公式](#) 有：

复三角函数在实数集上与实三角函数的定义完全一致。在复平面上拥有性质：

- 奇偶性：正弦函数是奇函数，余弦函数是偶函数。
- 三角恒等式：通常的三角恒等式都成立，例如平方和为，或者角的和差公式等。
- 周期性：正弦与余弦函数以 为基本周期。
- 零点：实正弦与实余弦函数的全体零点，构成了复正弦与复余弦函数的全体零点。这个推广没有引进新的零点。
- 模的无界性：复正弦与复余弦函数，模长可以大于任意给定的正数，不再像实正弦与实余弦函数一样被限制在 的范围内。

复数的三种形式

借助直角坐标系的视角以及极坐标系的视角，可以写出复数的三种形式。

复数的 **代数形式** 用于表示任意复数。

代数形式用于计算复数的加减乘除四个运算比较方便。

复数的 **三角形式** 和 **指数形式**，用于表示非零复数。

这两种形式用于计算复数的乘除两个运算以及后面的运算较为方便。如果只用高中见过的函数，可以使用三角形式。如

果引入了复指数函数，写成等价的指数形式会更加方便。

单位根

称 z 在复数意义下的解是 n 次复根。显然，这样的解有 n 个，称这 n 个解都是 n 次 **单位根** 或 **单位复根**（the n -th root of unity）。根据复平面的知识， n 次单位根把单位圆 n 等分。

设 $z = e^{2\pi i k/n}$ （即幅角为 $2\pi k/n$ 的单位复数），则 $z^n = 1$ 的解集表示为 $\{z^k\}_{k=0}^{n-1}$ 。

如果不加说明，一般叙述的 n 次单位根，是指从 1 开始逆时针方向的第一个解，即上述 z^0 ，其它解均可以用 z^k 的幂表示。

性质

单位根有三个重要的性质。对于任意正整数 n 和整数 k ：

推导留给读者自证。这三个性质在快速傅里叶变换中会得到应用。

本原单位根

为什么说，上述 n 个解都是 n 次单位根，而平时说的 n 次单位根一般特指第一个？

特指第一个，是为了在应用时方便。

在解方程的视角看来，满足 $z^n = 1$ 性质的不止一个，对于 z 的若干次幂也会满足性质。

称集合：

中的元素为 **本原单位根**。任意一个本原单位根 z ，与上述 z^k 具有相同的性质：对于任意的 m ， z^{km} 的 n 次幂不为 1 。因此，借助任意一个本原单位根，都可以生成全体单位根。

全体 n 次本原单位根共有 $\phi(n)$ 个。

编程语言中的复数

C 中的复数

在 C99 标准中，有 `<complex.h>` 头文件。

在 `<complex.h>` 头文件中，提供了 `double complex`、`float complex` 和 `long double complex` 三种类型。

算术运算符 '+'、'-'、'*' 和 '/'，可以用于浮点数和复数的任意混合。当表达式两端有一个为复数时，计算结果为复数。

头文件 `<complex.h>` 提供了虚数单位 `I`，引入此头文件时，大写字母 `I` 不可以作为变量名使用。

对于单个复数，`<complex.h>` 提供了若干操作：`creal` 函数用于提取实部，`cimag` 函数用于提取虚部，`cabs` 函数用于计算模，`carg` 函数用于计算辐角主值。

所有的函数根据类型不同，都有三个。例如 `creal` 函数有 `creal`、`crealf`、`creall` 三个，用于处理对应的 `double`、`float` 和 `long double` 三种类型。末尾什么都不带的默认处理 `double` 类型。以下所有函数均遵从此规律，不再特别说明。

这些函数返回值都是一般的浮点数。可以将普通浮点数直接赋值给复数，但是不可以将复数直接赋值给浮点数，而是需要使用上述提取操作。

函数 `conj` 用于计算共轭复数，返回值是复数。

函数 `cexp` 计算复指数，`clog` 计算对数主值，`csin` 计算正弦，`ccos` 计算余弦，`ctan` 计算正切。

函数 `cpow` 计算幂函数，`csqrt` 计算平方根，`casin` 计算反正弦，`cacos` 计算反余弦，`ctan` 计算反正切。这部分函数计算的全部都是多值函数的主值。

C++ 中的复数

在 C 里面的 `<Tctype.h>`，到 C++ 会变成 `<Tcctype>`，几乎所有的头文件遵从这个命名规律。

但是，`<Tcomplex.h>` 不遵守，C++ 没有 `<Tccomplex>` 头文件。C++ 的复数直接是 `<Tcomplex>`，并且装的东西和 C 完全不一样。

很有趣。这是因为，在 C++ 的第一个版本 C++98，即已经有了 `<Tcomplex>`，而 C 语言在 C99 才添加。

在 C++ 中，复数类型定义使用 `complex<LTfloat>`、`complex<LTdouble>` 和 `complex<LTlong double>`。由于面向对象的多态性，下面函数的名字都是唯一的，无需 `f` 或 `l` 的后缀。

一个复数对象拥有成员函数 `real` 和 `imag`，可以访问实部和虚部。

一个复数对象拥有非成员函数 `real`、`imag`、`abs`、`arg`，返回实部、虚部、模和辐角。

一个复数对象还拥有非成员函数：`norm` 为模的平方，`conj` 为共轭复数。

一个复数对象还拥有非成员函数 `exp`、`log`（底为 e 的对数主值）、`log10`（底为 10 的对数主值，C 中没有）、`pow`、`sqrt`、`sin`、`cos`、`tan`，含义与 C 中的含义相同。

在 C++14 及以后的版本中，定义了 [字面量运算符 `std::literals::complex\_literals::"if"`、`"i"`、`"il"`](#)。例如输入 `100if`、`100i` 和 `100il`，三者将分别返回 `std::complex<LTfloat>{0.0f, 100.0f}`、`std::complex<LTdouble>{0.0, 100.0}` 以及 `std::complex<LTlong double>{0.0l, 100.0l}`。这使得我们可以方便地书写形如 `auto z = 4 + 3i` 的复数声明。

参考资料与链接

- [Complex number - Wikipedia](#)
- [Euler's formula - Wikipedia](#)
- [Complex number arithmetic - cppreference.com](#)
- [std::complex - cppreference.com](#)

-
1. 有关欧拉公式的更多介绍，可以参考两个视频：[欧拉公式与初等群论](#)、[微分方程概论 - 第五章：在 3.14 分钟内理解](#)。↩
-

本页面最近更新：2023/12/19 13:50:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[CCXXLI](#), [CharlesWuQiushi](#), [chinggg](#), [Enter-tainer](#), [Great-designer](#), [HeRaNO](#), [iamtwz](#), [Ir1d](#), [mcendu](#), [megakite](#), [MegaOwler](#), [ouuan](#), [r-value](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [TSStudio](#), [Xeonacid](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用